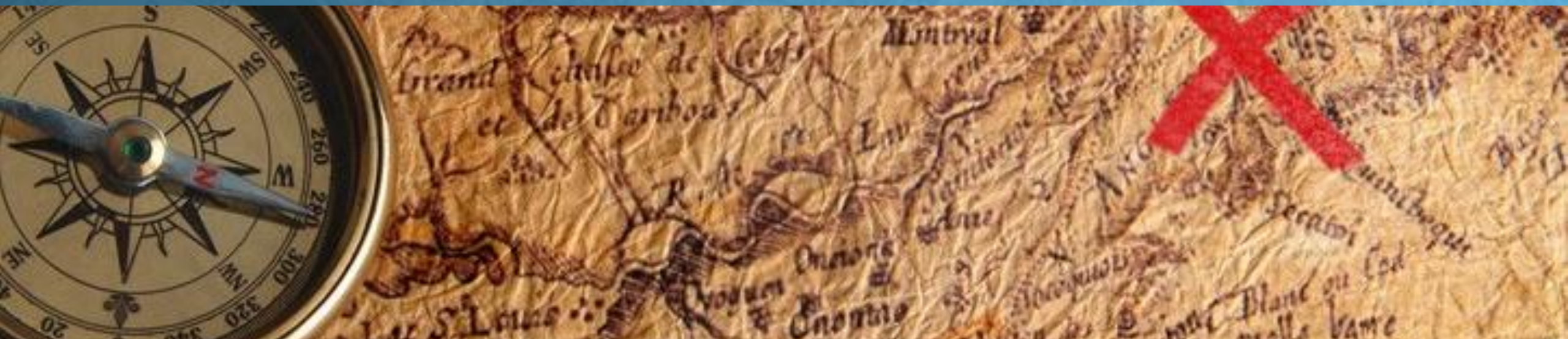


5. Clasificación

Aprendizaje supervisado



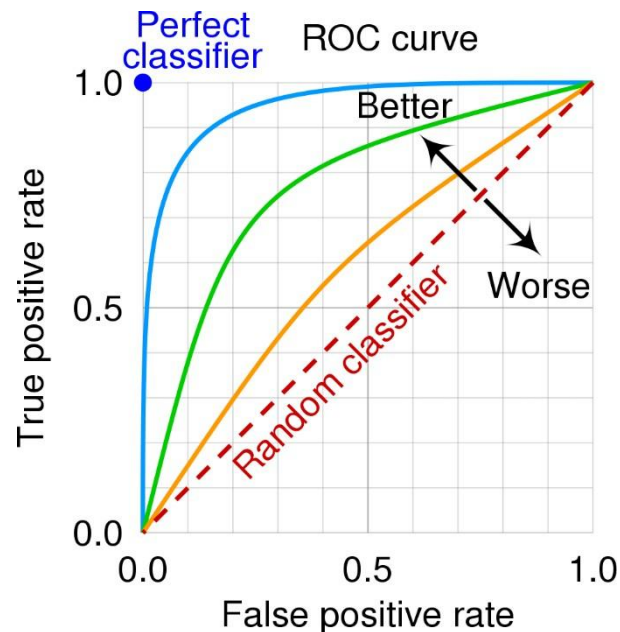
MD005 – Sistemas Basados en el Conocimiento

Jordina Torrents - jordina.torrents@ext.salle.url.edu

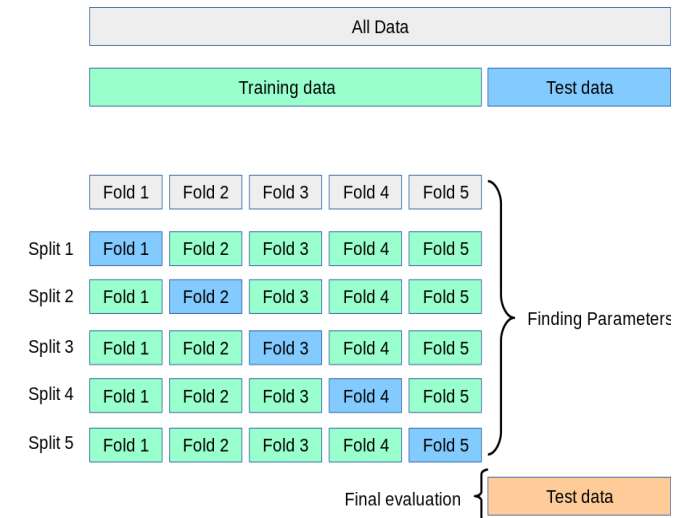
Recapitulemos

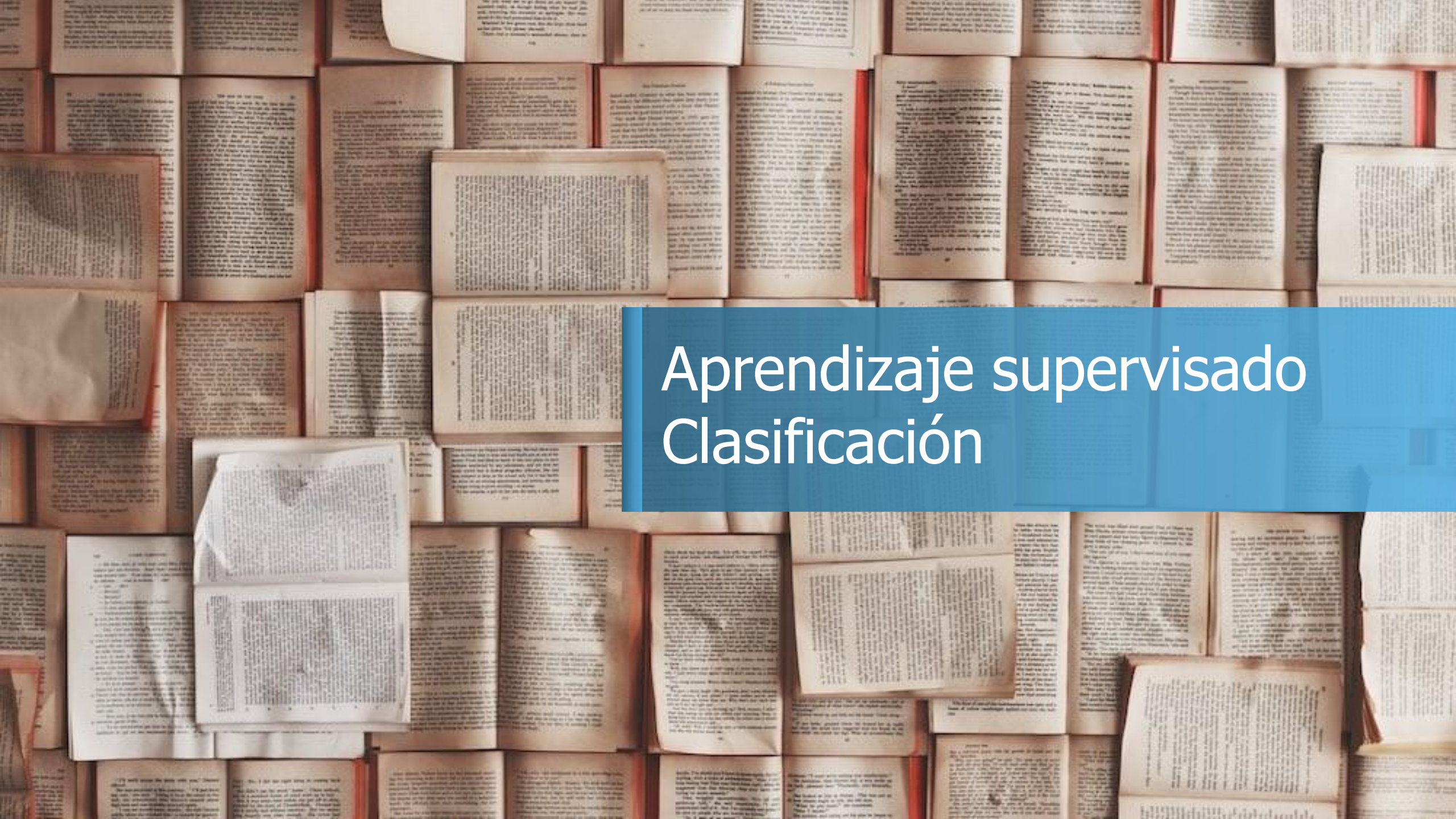
Métricas

- Características buscadas de los sistemas (generalidad, robustez, explicabilidad, determinista...)
- Clasificación de los algoritmos según el enfoque o el problema que resuelve.
- Métricas de clasificación y regresión.



	Predicted Positive	Predicted Negative	
Actual Positive	TP <i>True Positive</i>	FN <i>False Negative</i>	Sensitivity $\frac{TP}{(TP + FN)}$
Actual Negative	FP <i>False Positive</i>	TN <i>True Negative</i>	Specificity $\frac{TN}{(TN + FP)}$
	Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$





Aprendizaje supervisado Clasificación

¿Qué quiere decir clasificar?

Tipo de aprendizaje supervisado que busca predecir la categoría o clase de la muestra que estamos evaluando a partir de conocimiento extraído de muestras disponibles con anterioridad.



Tipos de clasificaciones

Hasta ahora hemos visto algoritmos de agrupación que o bien definíamos el número de grupos o bien las características para que el algoritmo lo defina por sí solo (Kmeans/DBSCAN).

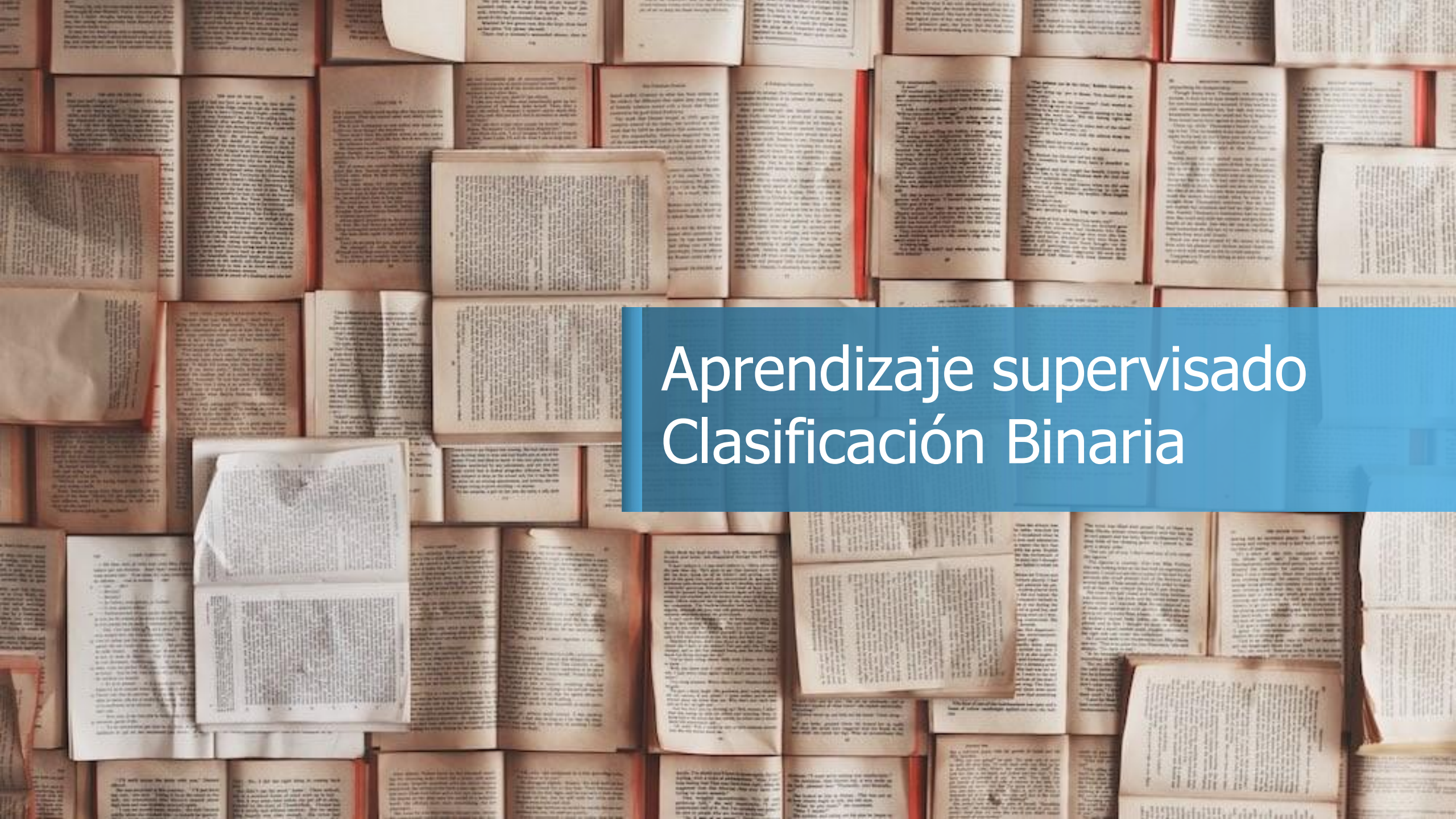
En los problemas de clasificación ya sabemos las clases a las que puede pertenecer y por lo tanto, se enfoca el problema de forma distinta principalmente preguntándonos:

¿Cuántas clases diferentes hay?

A partir de este punto, podremos descartar varios algoritmos y acotar los que vayamos a utilizar en función de la complejidad del problema junto con la cantidad de datos disponibles.

Tipos de clasificaciones

- **Binaria:** Clasificación con la menor complejidad, únicamente identifica si pertenece a una clase u a otra (0/1).
 - **Regresión Logística**
 - **Árboles de decisión**
 - **K-NN (nearest neighbors)**
 - **Support Vector Machines**
 - **Naïve Bayes**
- **Multiclase:** Generalización del tipo binario, identificamos a qué grupo pertenece una muestra pero esta vez, entre varios grupos (>2).
 - **Árboles de decisión**
 - **K-NN**
 - **Support Vector Machines**
- **Multiobjetivo:** Versión en la que se predicen diferentes clasificaciones diferentes a la vez, por ejemplo si utilizásemos datos de pingüinos y la predicción fuese a la vez el sexo y la isla a la que pertenecen.



Aprendizaje supervisado

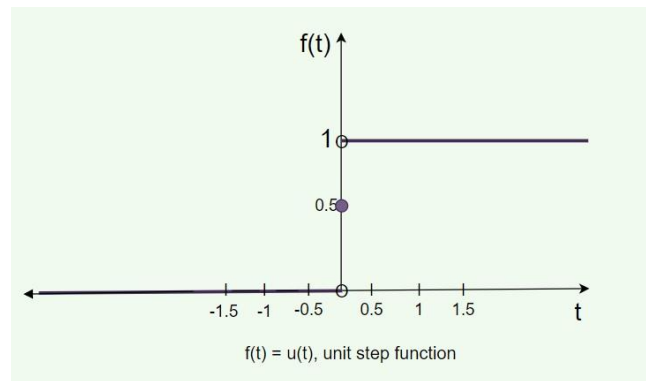
Clasificación Binaria

Clasificación binaria

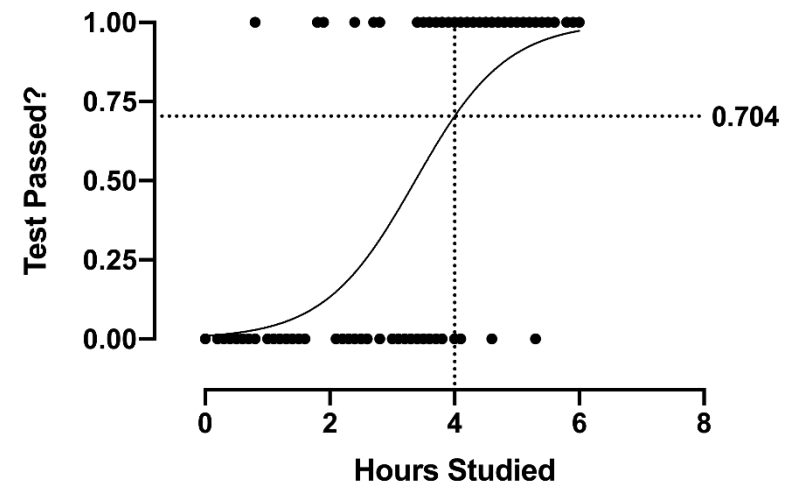
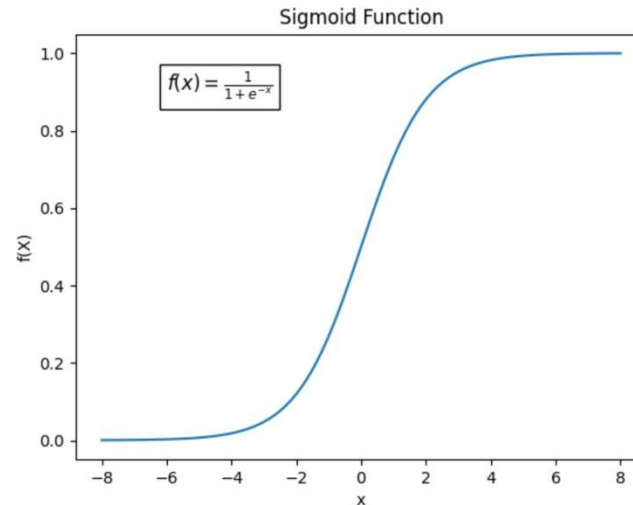
Tipo de clasificación más sencilla/interpretable y que es a la vez un caso característico de la clasificación multiclase y consiste en clasificar entre 2 opciones posibles (siempre codificables en 0 y 1).

El algoritmo más sencillo para realizar dicha clasificación es la **regresión logística** pero no es la única forma de realizarla, tan sólo necesitamos una función con la que podamos interpretar 2 estados y que un **valor límite** nos defina cuál es el **valor de transición**.

- **Valor límite:** es el umbral o número que se usa para decidir entre dos estados (e.g., 0.5, usando una probabilidad del modelo de 0 a 1).
- **Valor de transición:** es el punto en el que el resultado cambia de un estado a otro al superar (o no) ese valor límite.



Escalón unitario de Laplace



Regresión Logística

IDEA:

- La regresión logística es la regresión lineal (la regresión más simple) aplicada a la clasificación. Por tanto, sería: *regresión lineal + transformación sigmoide + decisión binaria*.

En una regresión lineal ¿qué ocurre cuando la variable respuesta es binaria, solo dos posibles valores?

PROBLEMAS (considerando la variable objetivo codificada en 0 y 1):

- Al generar la recta (hiperplano) se pueden obtener **valores distintos de 0 y 1**, contradiciendo la definición de la variable binaria.
- Si interpretamos las predicciones como **probabilidades**, estas **podrían superar el rango [0,1]**, lo que resulta inconsistente con la definición de probabilidad.



Regresión Logística

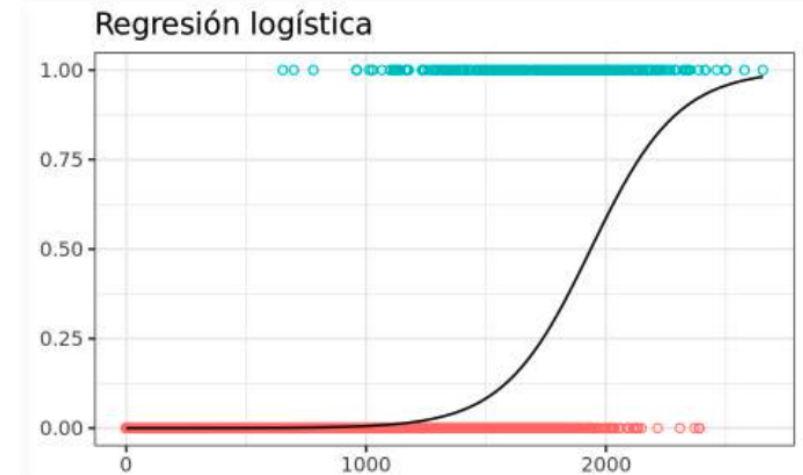
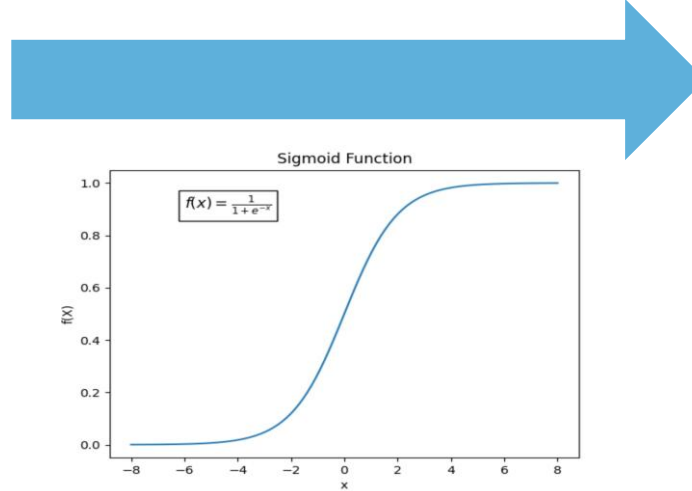
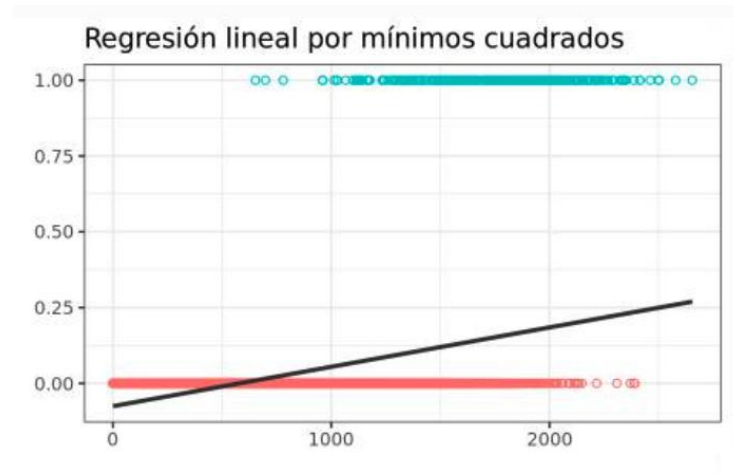
SOLUCIÓN:

- **Transformar** el valor devuelto por la regresión lineal con una función cuyo resultado esté acotado $[0,1]$.
- La **función logística** es la más común (**función sigmoide**).

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}$$

$$\sigma(y) = \frac{1}{1 + e^{-y}}$$

$$P(y = 1 | X = x) = \frac{e^{\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p}}$$



Regresión Logística

Nuevo PROBLEMA:

- ¿Como ajustamos ahora los parámetros de la regresión?

Nueva SOLUCIÓN:

1. Considerando la función sigmoide como una **función coste**, pudiendo calcular el **error** durante el entrenamiento (predicción – valor real). Los parámetros se optimizan iterando con el algoritmo *back-propagation* (método que usan las redes neuronales para aprender de sus errores: 1) la red hace una predicción, luego compara el resultado con la respuesta correcta para calcular el error, y 2) “propaga” ese error hacia atrás a través de todas sus capas para ajustar los pesos de las conexiones. De esta manera, cada conexión se modifica ligeramente para que la próxima predicción sea más precisa, permitiendo que la red mejore gradualmente con el tiempo).
2. **Transformando** la regresión logística para poder aplicar una **regresión lineal simple**:

$$P(y = 1|X = x) = \frac{e^{\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p}} \xleftrightarrow{\text{Transformación matemática}} \ln\left(\frac{p(y = 1|X = x)}{p(y = 0|X = x)}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Regresión Logística

Interpretación del modelo:

- **Intercepto (β_0):** Representa el logaritmo de los *odds* cuando todos los predictores son cero. Es el punto de partida del modelo. Es la **probabilidad de que el resultado sea “1”** cuando todos los demás *features* (predictores) son cero. Es la probabilidad base antes de considerar cualquier efecto de los predictores. Transformado a probabilidad:

$$P = \frac{e^{\beta_0}}{1 + e^{\beta_0}} \quad \longleftrightarrow \quad P(y = 1 | X = x) = \frac{e^{\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p}}$$

Los *odds* (probabilidad relativa) son una forma de expresar la probabilidad de que algo ocurra en comparación con que no ocurra.

Coeficientes (β_p): Indican el cambio en el logaritmo de los *odds* por cada incremento unitario en el predictor x_p , manteniendo constantes los demás. Cada coeficiente muestra **cómo cambia la probabilidad** de que el resultado sea “1” cuando aumentas en 1 unidad ese predictor específico, **sin cambiar los demás predictores**.

$$\frac{\text{Odds}(x_p + 1)}{\text{Odds}(x_p)} = e^{\beta_p} \quad \longleftrightarrow \quad \Delta \ln \left(\frac{P(y = 1 | x)}{1 - P(y = 1 | x)} \right) = \beta_p \quad \longleftrightarrow \quad \ln \left(\frac{p(y = 1 | X = x)}{p(y = 0 | X = x)} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

- Si $\beta_p > 0$: Los *odds* aumentan.
- Si $\beta_p < 0$: Los *odds* disminuyen.
- Si $\beta_p = 0$: Los *odds* permanecen iguales.

Regresión Logística: Cuándo y por qué

El algoritmo tiene varias características buscadas en los algoritmos de Machine Learning (ML):

- **Interpretabilidad:** Podemos analizar qué importancia tiene cada variable.
- **Separabilidad lineal:** Para su funcionalidad correcta, los grupos han de ser separables de forma lineal (como con K-Means).
- **Eficiencia computacional:** Combinación lineal de las variables a modo de sistema de ecuaciones.
- **Resultado probabilístico:** Resultado interpretable como una probabilidad y no una identificación como tal.
- **No asume normalidad:** A diferencia de muchos otros algoritmos de ML, no necesita que las distribuciones sean normales para su buen funcionamiento.

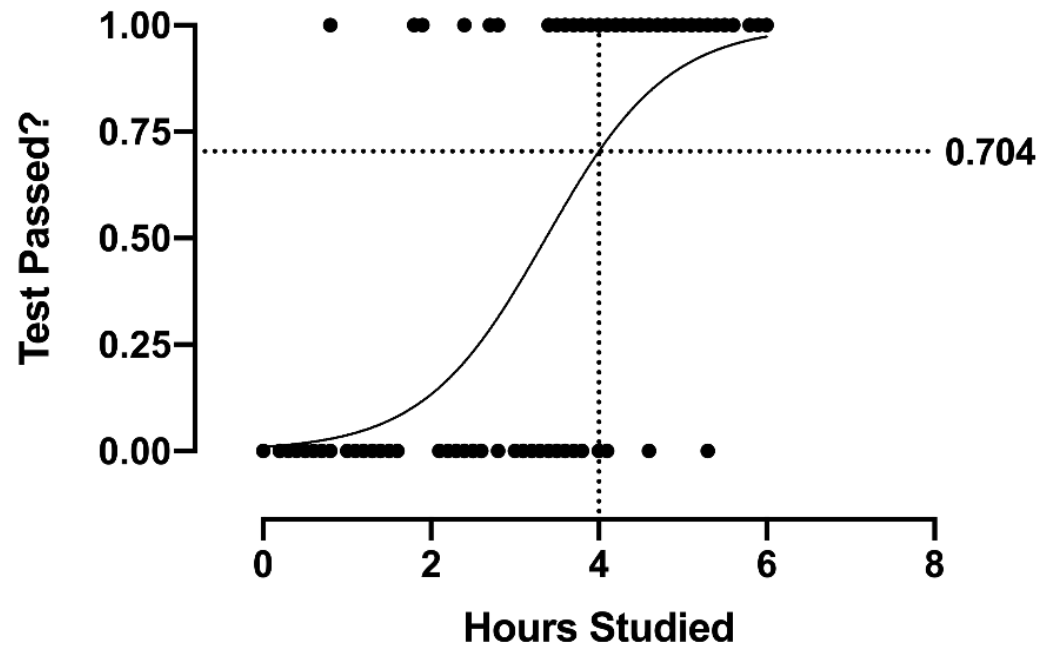
Regresión Logística: Puntos débiles

A su vez, no hay algoritmos que funcionen en todas las situaciones y que sean excelentes, todos tienen detalles que nos harán escoger otro algoritmo o procesar los datos de una forma concreta.

- **Separabilidad lineal:** Depende en gran medida, si los grupos no tienen la cualidad (por lo menos de forma parcial), no será funcional. Al igual que si los límites entre grupos (**decision boundaries**) tienen formas complejas.
- **Variables de entrada:** Tener un gran número de variables de entrada dificulta (y mucho) el funcionamiento del algoritmo, selección de variables importantes y las técnicas de reducción de dimensionalidad (e.g., PCA, UMAP) serán aliadas:
 - **PCA (Análisis de Componentes Principales):** transforma datos complejos en un conjunto menor de variables llamadas componentes principales, conservando la mayor parte de la varianza. Simplifica los datos y facilita su visualización y análisis. Se basa en encontrar direcciones lineales que expliquen la mayor variabilidad de los datos.
 - **UMAP (Uniform Manifold Approximation and Projection):** captura la estructura global y local de los datos, ideal para visualización en 2D o 3D. Se enfoca en conservar las relaciones de proximidad entre los puntos en el espacio original. Funciona bien con datos no lineales y estructuras complejas, mostrando agrupamientos de manera clara.

Regresión Logística: Ejemplos

- Aprobar una asignatura en función de 'algo'.
- A partir de datos médicos identificar si un paciente tiene o no una enfermedad.
- Definir si un cliente está en riesgo y por lo tanto, deja de ser cliente (churn)
- Evaluar piezas en procesos de fabricación: ¿pasan o no el control de calidad?





iAl Código! LogisticRegression

Máquinas de Soporte Vectorial (SVM) - Conceptos

SVM es un modelo que representa a los puntos de muestra en el espacio, **separando las clases** a 2 (o N) espacios lo más amplios posibles **mediante un hiperplano de separación**.

- **Hiperplano:** En geometría es una extensión del concepto de plano
 - Plano: es un objeto ideal que solo posee dos dimensiones, y contiene infinitos puntos y rectas,
 - En un espacio unidimensional (como una recta), un hiperplano es un punto: divide una línea en dos líneas. En un espacio bidimensional (como el plano xy), un hiperplano es una recta: divide el plano en dos mitades.

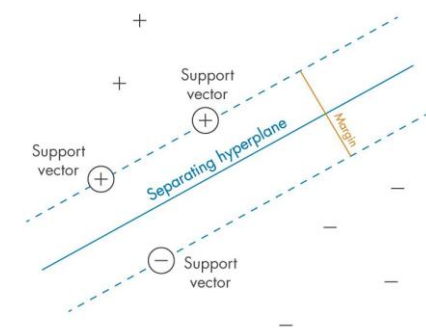
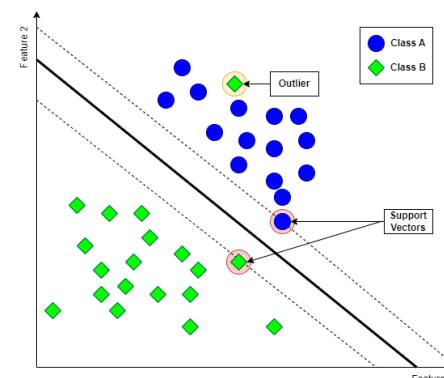


Los vectores de soporte son **los puntos de datos más importantes** que determinan la posición del **hiperplano de separación** en un SVM.

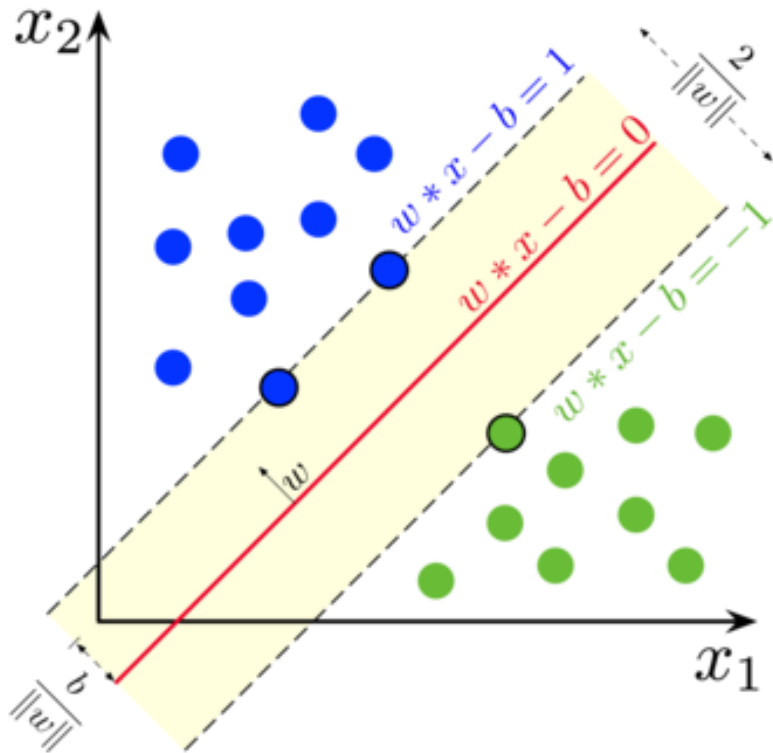
- Son los puntos **más cercanos al margen**.
- No todos los puntos influyen en la decisión, solo estos puntos críticos.

Para qué sirven:

- Definen el **margen máximo** entre clases.
- El SVM **solo depende de ellos**, no de todos los datos.
- Son los que “soportan” el hiperplano, de ahí su nombre.



Máquinas de Soporte Vectorial



1. Hiperplano de decisión (línea roja central):

$$w \cdot x - b = 0$$

- Es el **límite** que separa las dos clases (azules y verdes).
- Aquí, w es el **vector normal** al hiperplano (apunta perpendicularmente a él).
- b es el **sesgo** que desplaza el hiperplano en el espacio.
- Todos los puntos que cumplen esta ecuación están **exactamente sobre la línea central**.

2. Márgenes (líneas punteadas):

- Límite superior (azul):

$$w \cdot x - b = 1$$

- Límite inferior (verde):

$$w \cdot x - b = -1$$

- Estas dos líneas definen el **margen máximo** alrededor del hiperplano.
- La **distancia entre ellas** es:

$$\text{margen} = \frac{2}{\|w\|}$$

donde $\|w\|$ es la norma del vector w .

3. Support vectors (puntos sobre las líneas de margen):

- Son los **puntos** que "tocan" las líneas punteadas (azules y verdes).
- Son **críticos** porque definen el hiperplano.
- Cambiar un support vector **cambia la posición del hiperplano**, mientras que los demás puntos no afectan.

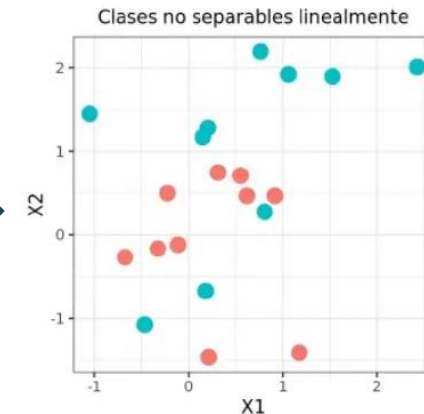
4. Vector normal w :

- Muestra la **dirección perpendicular al hiperplano**.
- Su magnitud está relacionada con el **ancho del margen**: margen más ancho $\rightarrow$ vector más pequeño.

Soft Margin en SVM

En un SVM clásico (**hard margin**), el modelo busca separar los datos con un **hiperplano** de manera que **ningún punto quede del lado “equivocado”**.

- Esto funciona si los datos son **perfectamente separables**.
- Pero en la vida real, los datos suelen estar **mezclados o tener ruido**.

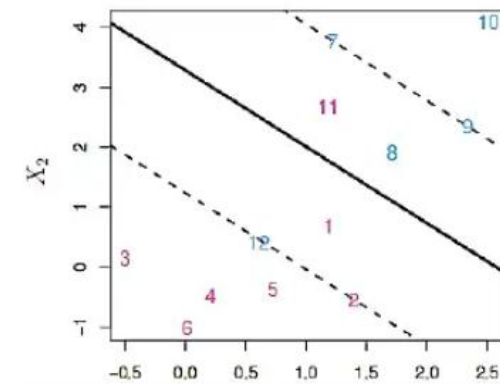


1. Concepto principal:

- Permite que **algunos puntos se “equivoken”** o queden dentro del margen de separación.
- El SVM intenta **maximizar el margen** mientras **penaliza las violaciones**.
- Esto evita que el modelo sea demasiado estricto y **sobreajuste** los datos.

2. Cómo funciona:

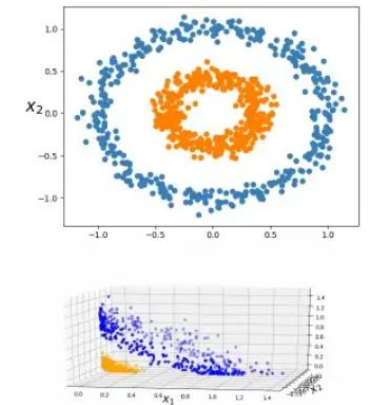
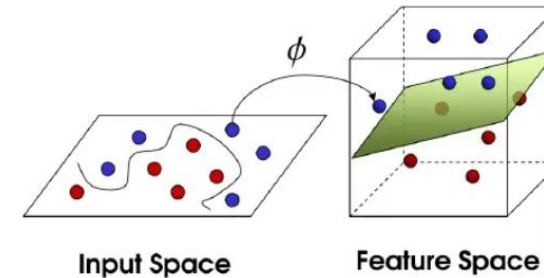
- Se introduce una variable llamada **slack variable** (ξ_i) para cada punto que mide **cuánto viola el margen**.
- Se define un parámetro **C** que controla el **equilibrio entre margen amplio y errores**:
 - C grande $\rightarrow$ el modelo **prefiere no cometer errores**, margen más estrecho.
 - C pequeño $\rightarrow$ el modelo **acepta más errores**, margen más amplio.



¿Qué es una función Kernel?

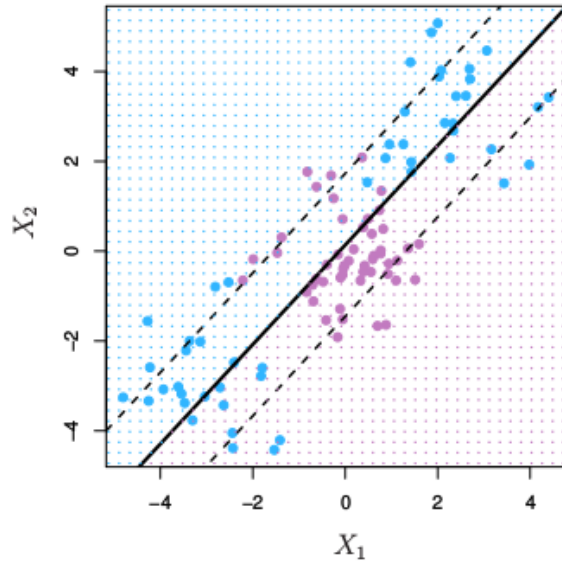
Un **kernel** es una función matemática que **transforma los datos originales** de tal manera que, aunque en su forma original no se puedan separar con una línea o un plano, en un nuevo espacio sí se puedan separar fácilmente.

- En SVM, el objetivo es encontrar un **hiperplano** que divida los datos en diferentes clases.
- Cuando los datos no son linealmente separables (por ejemplo, están mezclados o forman círculos concéntricos), un **kernel “mapea” los datos a un espacio de mayor dimensión** donde se vuelven separables.
- Lo importante: esta transformación se hace **sin necesidad de calcular explícitamente cada nueva dimensión**, gracias a lo que se llama “**truco del kernel**” (*kernel trick*). Esto ahorra mucho cálculo y hace posible separar datos muy complejos.
- Así, SVM puede resolver problemas **no lineales** con la misma eficiencia que en problemas lineales.



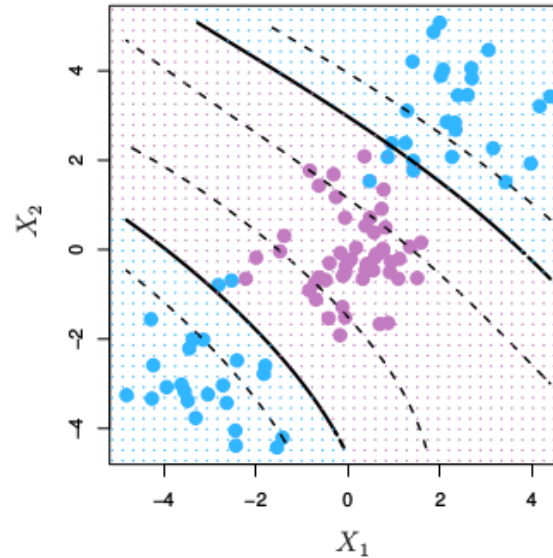
Tipos de Kernels

Kernel Lineal



$$K(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$$

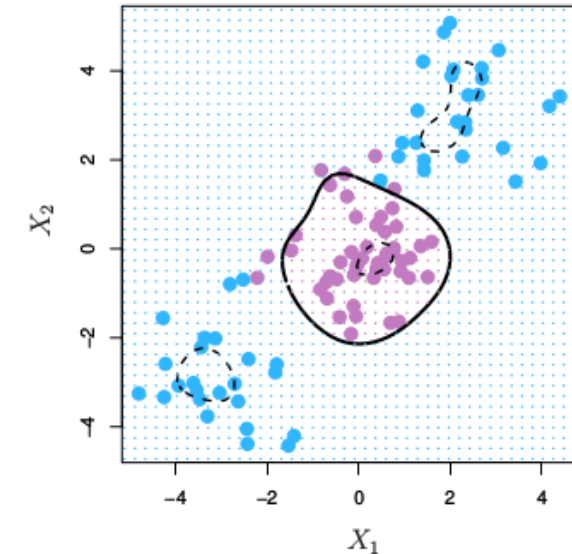
Kernel Polinómico



$$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + c)^d$$

Cuando se emplea $d = 1$ y $c = 0$, el resultado es el mismo que el de un kernel lineal. Si $d > 1$, se generan límites de decisión no lineales, aumentando la no linealidad a medida que aumenta d . No suele ser recomendable emplear valores de $d > 5$ por problemas de overfitting.

Kernel Gaussian (RBF)



$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$$

El valor de γ controla el comportamiento del kernel, cuando es muy pequeño, el modelo final es equivalente al obtenido con un kernel lineal, a medida que aumenta su valor, también lo hace la flexibilidad del modelo.



Aprendizaje supervisado

Clasificación Multiclase

Clasificación multiclase

Caso general en el que la clasificación no se realiza entre un grupo A y un grupo B sino que hay por lo menos 3 de ellos, imposibilitando el uso de la regresión logística para clasificar

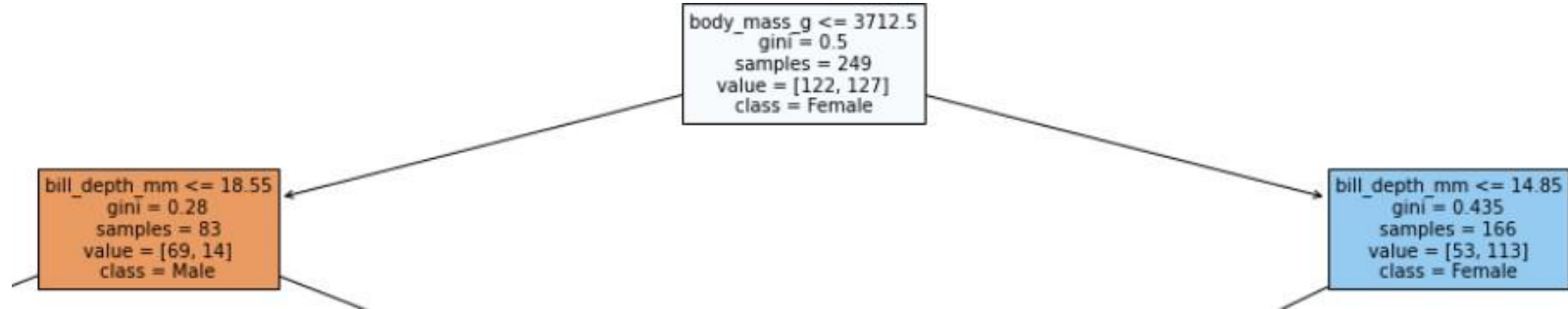
- **Árboles de decisión:** Con la estructura familiar de los espacios de búsqueda, identifica la variable que reduce la impureza del nodo para segregar las muestras y llegar a una clasificación.
- **K-NN:** Te basas en los vecinos (K-nearest neighbors) y en esencia, te asigna la clase de los vecinos de tu cercanía.

- Un **nodo de decisión** está asociado a uno de los atributos y tiene 2 o más ramas que salen de él, cada una de ellas representando los posibles valores que puede tomar el atributo asociado. De alguna forma, un nodo de decisión es como una pregunta que se le hace al ejemplo analizado, y dependiendo de la respuesta que de, el flujo tomará una de las ramas salientes.

```

graph TD
    Root["body_mass_g <= 3712.5  
gini = 0.5  
samples = 268  
value = [222, 127]  
class = Female"]
    
    Root --> L1L["bdl_depth_mean <= 34.9  
gini = 0.28  
samples = 63  
value = [19, 14]  
class = Male"]
    Root --> L1R["bdl_depth_mean > 34.9  
gini = 0.48  
samples = 164  
value = [53, 111]  
class = Female"]
    
    L1L --> L2L1["bdl_length_mean <= 68.1  
gini = 0.46  
samples = 41  
value = [7, 2]  
class = Male"]
    L1L --> L2L2["bdl_length_mean > 68.1  
gini = 0.46  
samples = 22  
value = [12, 10]  
class = Male"]
    
    L2L1 --> L3L1["bdl_length_mean <= 44.9  
gini = 0.3  
samples = 13  
value = [3, 1]  
class = Male"]
    L2L1 --> L3L2["bdl_length_mean > 44.9  
gini = 0.3  
samples = 28  
value = [4, 18]  
class = Male"]
    
    L2L2 --> L3L3["bdl_length_mean <= 40.1  
gini = 0.3  
samples = 19  
value = [3, 16]  
class = Male"]
    L2L2 --> L3L4["bdl_length_mean > 40.1  
gini = 0.3  
samples = 3  
value = [9, 3]  
class = Male"]
    
    L1R --> L2R1["bdl_length_mean <= 39.1  
gini = 0.48  
samples = 59  
value = [4, 55]  
class = Female"]
    L1R --> L2R2["bdl_length_mean > 39.1  
gini = 0.48  
samples = 105  
value = [17, 88]  
class = Female"]
    
    L2R1 --> L3R1["bdl_depth_mean <= 19.5  
gini = 0.27  
samples = 6  
value = [1, 5]  
class = Male"]
    L2R1 --> L3R2["bdl_depth_mean > 19.5  
gini = 0.27  
samples = 53  
value = [1, 52]  
class = Male"]
    
    L2R2 --> L3R3["bdl_depth_mean <= 14.8  
gini = 0.176  
samples = 125  
value = [12, 113]  
class = Female"]
    L2R2 --> L3R4["bdl_depth_mean > 14.8  
gini = 0.176  
samples = 80  
value = [17, 63]  
class = Female"]
    
    L3R3 --> L4R31["bdl_length_mean <= 38.0  
gini = 0.176  
samples = 28  
value = [4, 24]  
class = Female"]
    L3R3 --> L4R32["bdl_length_mean > 38.0  
gini = 0.176  
samples = 97  
value = [1, 96]  
class = Female"]
    
    L3R4 --> L4R41["bdl_length_mean <= 35.8  
gini = 0.176  
samples = 67  
value = [1, 66]  
class = Female"]
    L3R4 --> L4R42["bdl_length_mean > 35.8  
gini = 0.176  
samples = 13  
value = [1, 12]  
class = Female"]
    
    L3L3 --> L4L31["bdl_depth_mean <= 14.8  
gini = 0.2  
samples = 11  
value = [1, 10]  
class = Male"]
    L3L3 --> L4L32["bdl_depth_mean > 14.8  
gini = 0.2  
samples = 17  
value = [3, 14]  
class = Male"]
    
    L3L4 --> L4L41["bdl_depth_mean <= 14.8  
gini = 0.2  
samples = 1  
value = [1, 0]  
class = Male"]
    L3L4 --> L4L42["bdl_depth_mean > 14.8  
gini = 0.2  
samples = 2  
value = [2, 0]  
class = Male"]
    
    L4L31 --> L5L311["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 5  
value = [1, 4]  
class = Male"]
    L4L311 --> L6L3111["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 2  
value = [1, 1]  
class = Male"]
    L4L311 --> L6L3112["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 3  
value = [0, 3]  
class = Male"]
    
    L4L32 --> L5L321["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 10  
value = [1, 9]  
class = Male"]
    L4L32 --> L5L322["bdl_length_mean > 34.9  
gini = 0.3  
samples = 7  
value = [2, 5]  
class = Male"]
    
    L4L41 --> L5L411["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L4L42 --> L5L421["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    
    L5L311 --> L6L3111["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 2  
value = [1, 1]  
class = Male"]
    L5L311 --> L6L3112["bdl_length_mean > 34.9  
gini = 0.3  
samples = 3  
value = [0, 3]  
class = Male"]
    
    L5L321 --> L6L3211["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 5  
value = [1, 4]  
class = Male"]
    L5L321 --> L6L3212["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 5  
value = [1, 4]  
class = Male"]
    
    L5L322 --> L6L3221["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 2  
value = [1, 1]  
class = Male"]
    L5L322 --> L6L3222["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 5  
value = [1, 4]  
class = Male"]
    
    L5L411 --> L6L4111["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L5L411 --> L6L4112["bdl_length_mean > 34.9  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L5L421 --> L6L4211["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L5L421 --> L6L4212["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L6L3111 --> L7L31111["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L6L3111 --> L7L31112["bdl_length_mean > 34.9  
gini = 0.3  
samples = 1  
value = [0, 1]  
class = Male"]
    
    L6L3112 --> L7L31121["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 2  
value = [1, 1]  
class = Male"]
    L6L3112 --> L7L31122["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 1  
value = [0, 2]  
class = Male"]
    
    L6L3211 --> L7L32111["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 5  
value = [1, 4]  
class = Male"]
    L6L3211 --> L7L32112["bdl_length_mean > 34.9  
gini = 0.3  
samples = 5  
value = [1, 4]  
class = Male"]
    
    L6L3221 --> L7L32211["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 2  
value = [1, 1]  
class = Male"]
    L6L3222 --> L7L32221["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 5  
value = [1, 4]  
class = Male"]
    
    L6L3222 --> L7L32222["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L6L4111 --> L7L41111["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L6L4111 --> L7L41112["bdl_length_mean > 34.9  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L6L4211 --> L7L42111["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L6L4211 --> L7L42112["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L6L4212 --> L7L42121["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L6L4212 --> L7L42122["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L7L31111 --> L8L311111["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L7L31111 --> L8L311112["bdl_length_mean > 34.9  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L7L31112 --> L8L311121["bdl_depth_mean <= 14.8  
gini = 0.3  
samples = 1  
value = [1, 0]  
class = Male"]
    L7L31112 --> L8L311122["bdl_depth_mean > 14.8  
gini = 0.3  
samples = 0  
value = [0, 0]  
class = Male"]
    
    L7L31121 --> L8L311211["bdl_length_mean <= 34.9  
gini = 0.3  
samples = 2  
value = [1, 1]  
class = Male"]
    L7L31121 --> L8L311212["bdl_length_mean > 34.9  
gini = 0.3  
samples = 1  
value = [0, 2]  
class = Male"]
    
    L7L31
```

Clasificación multiclase: Árboles de clasificación

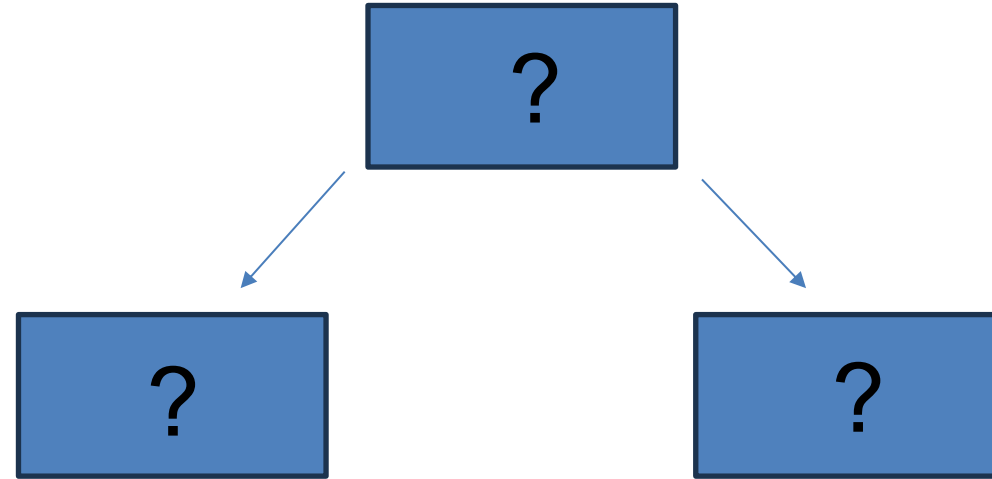


En la visualización podemos extraer **información** extremadamente útil para nuestro problema:

- **Variable y límite de corte** con el que se separa el nodo al siguiente nivel.
- **Criterio:** Métrica de evaluación, nos indica qué tan contaminado está el nodo entre las diferentes clases.
- Muestras que hay en el nodo y cómo se reparten entre las clases.
- A qué clase asignaría ese nodo si fuese un nodo hoja.

Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes



El **criterio** es la **regla que usa el árbol para decidir cómo dividir los datos** en cada nodo. Mide **qué tan mezcladas (contaminadas)** están las clases dentro de un nodo. Un nodo está **poco contaminado** si casi todos los datos pertenecen a una sola clase, y **muy contaminado** si hay muchas clases mezcladas.

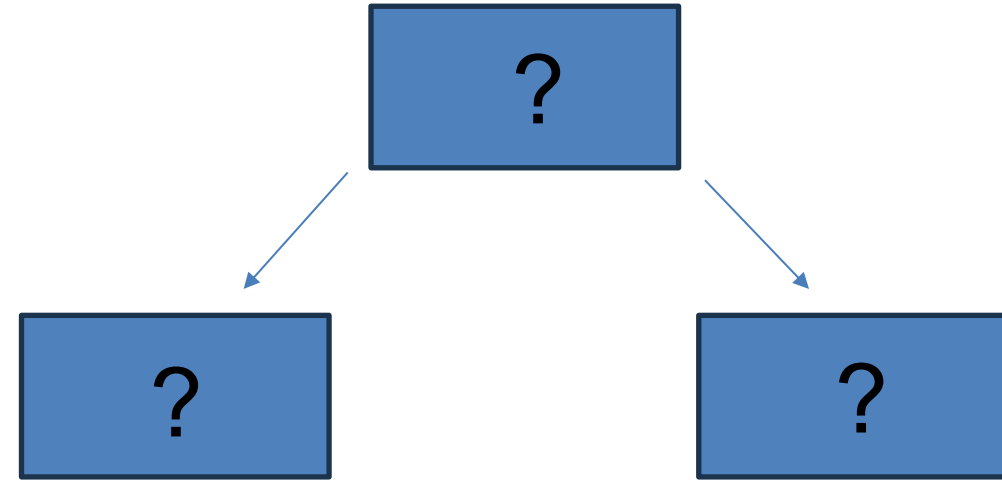
Los tres criterios más comunes son:

- **Gini**: mide qué tan probable es clasificar mal un dato al azar; valores bajos indican nodos más “puros”.
- **Entropy**: mide el desorden o incertidumbre del nodo; cuanto menor es, más clara es la clase dominante.
- **Log loss**: penaliza mucho las predicciones incorrectas y se usa más cuando el modelo devuelve probabilidades.

El **criterio** indica **qué tan buena es una división**, y el árbol elige la que **reduce más la contaminación del nodo**.

Clasificación multiclase: Árboles de clasificación

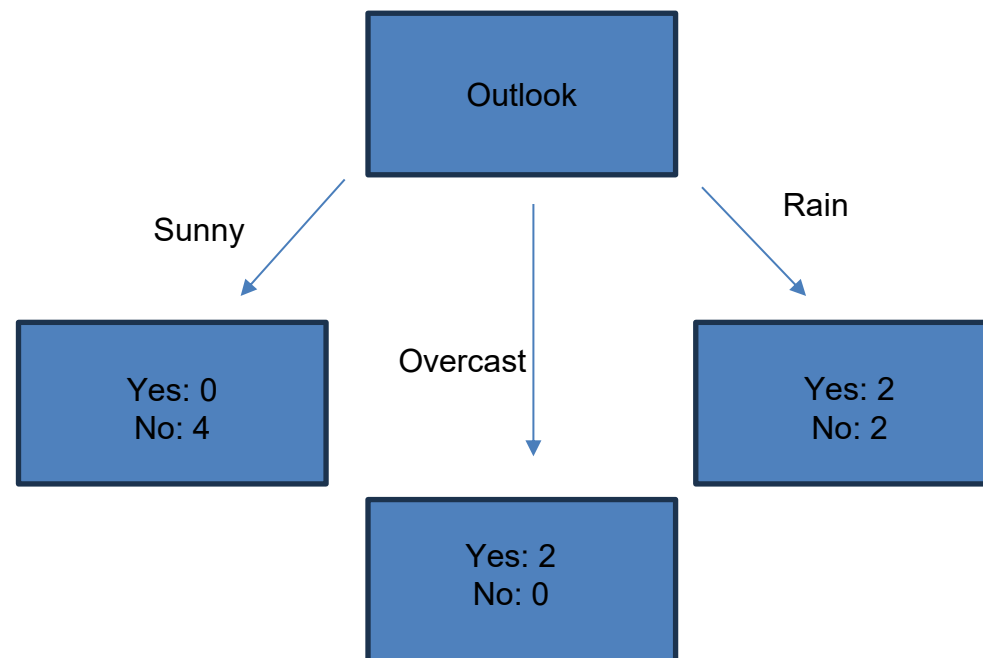
Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes



Gini Impurity $= 1 - \sum_j p_j^2$

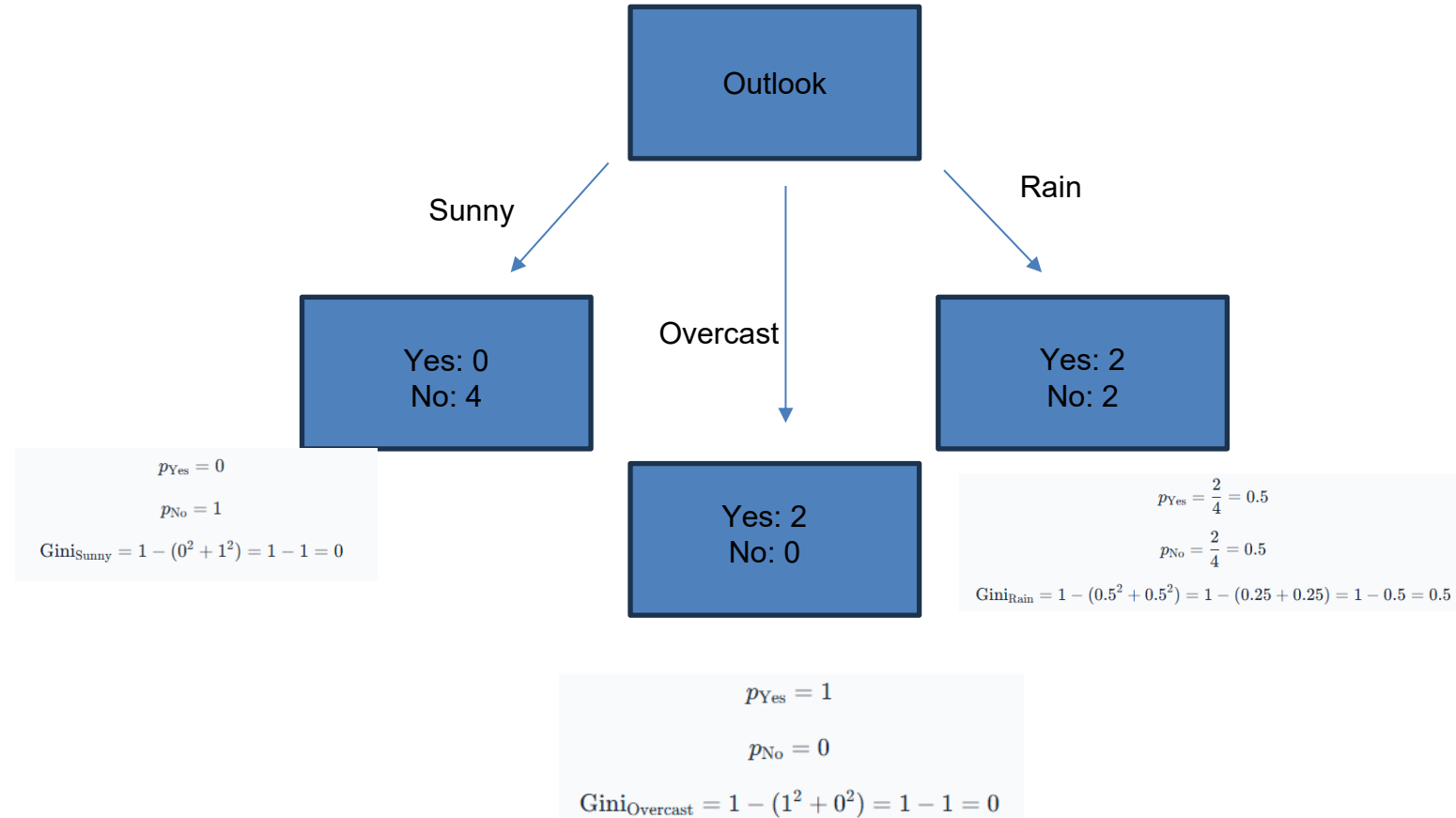
Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes



Clasificación multiclase: Árboles de clasificación

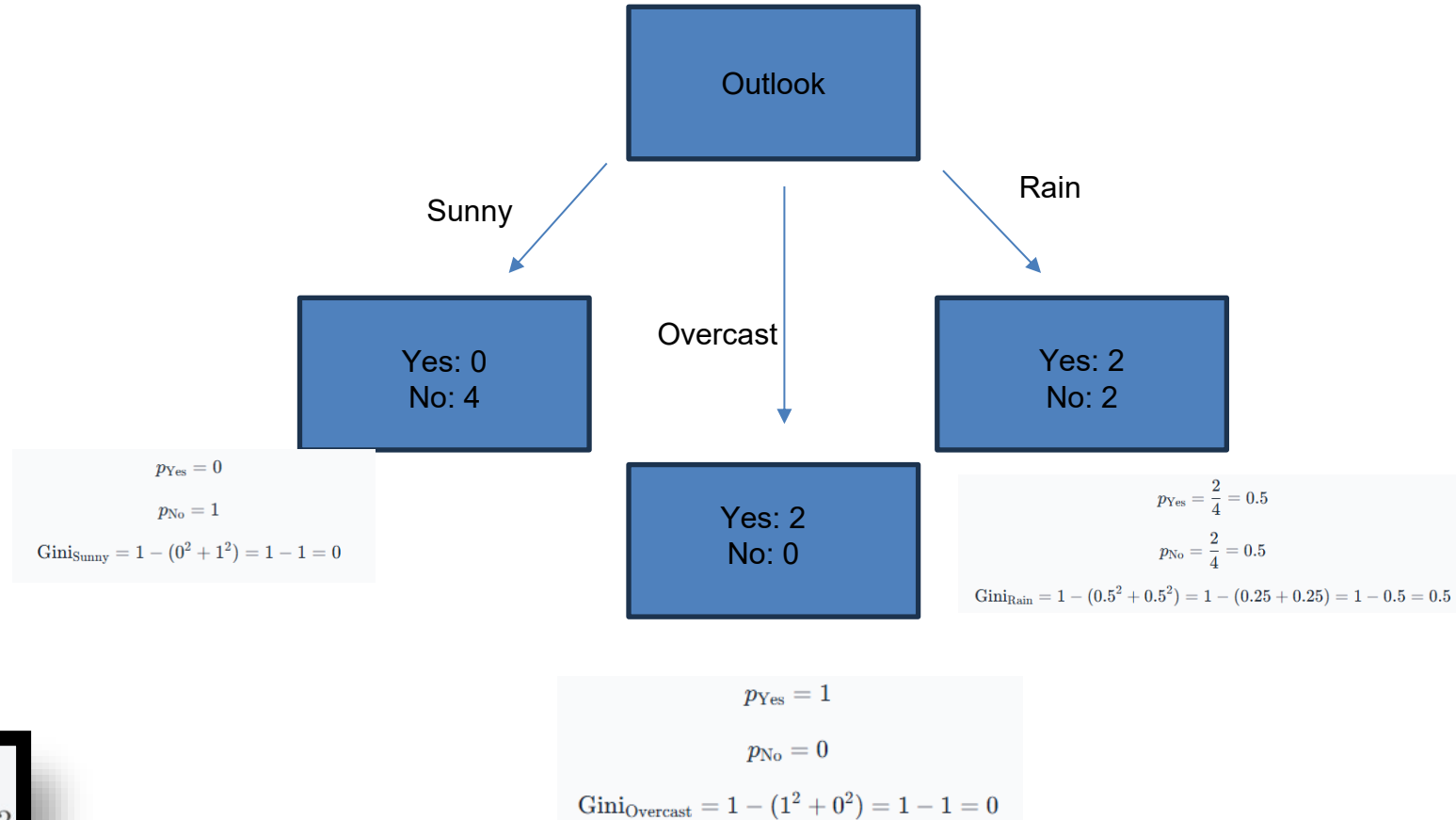
Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes



Clasificación multiclase: Árboles de clasificación

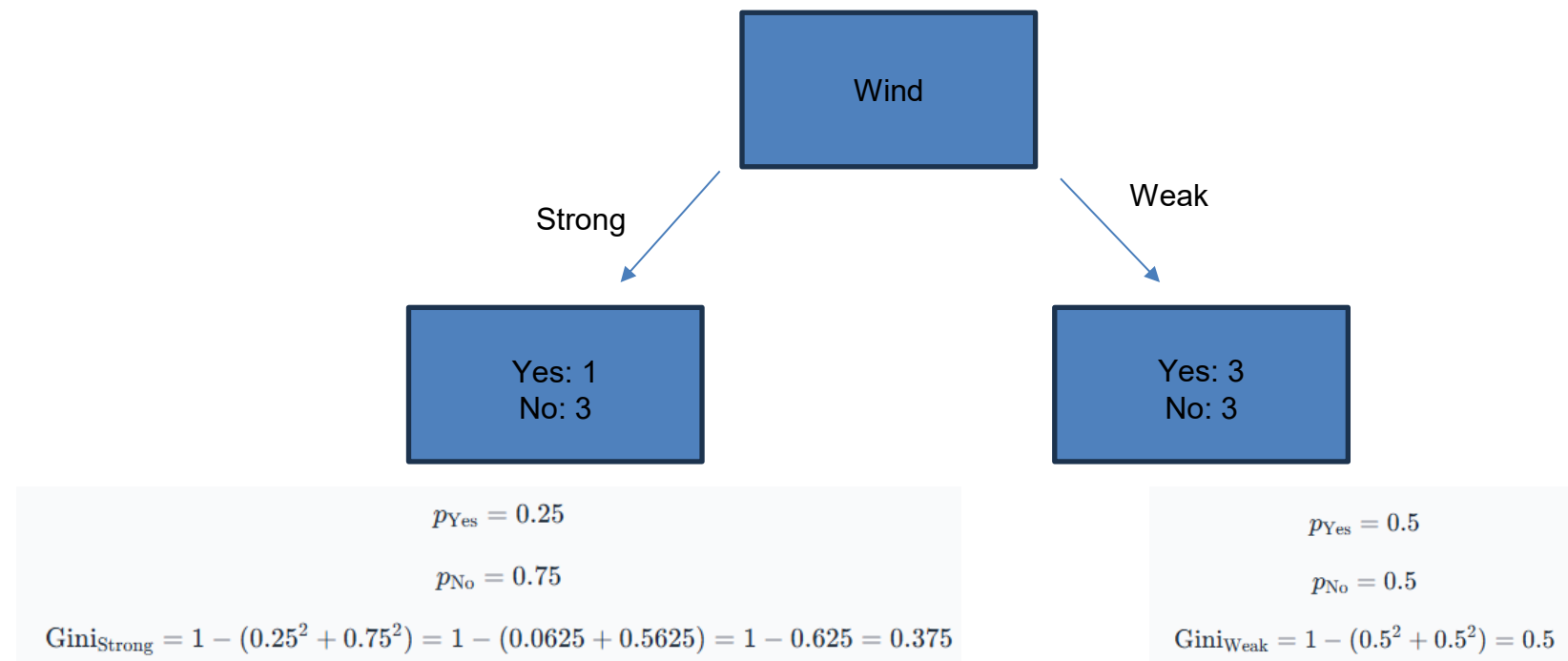
Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes

$$\text{Gini}_{\text{Outlook}} = \left(\frac{4}{10} \times 0 \right) + \left(\frac{2}{10} \times 0 \right) + \left(\frac{4}{10} \times 0.5 \right) = 0 + 0.2$$



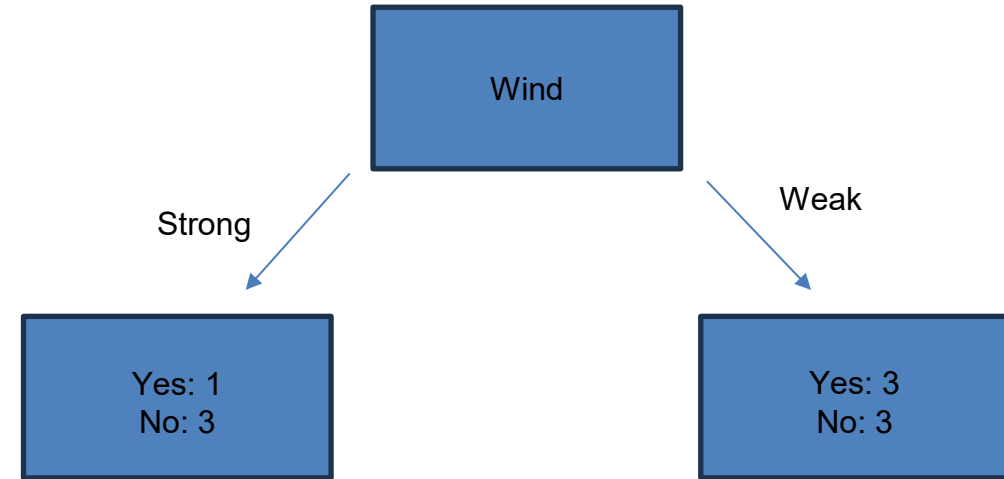
Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes



Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes



$$p_{\text{Yes}} = 0.25$$

$$p_{\text{No}} = 0.75$$

$$\text{Gini}_{\text{Strong}} = 1 - (0.25^2 + 0.75^2) = 1 - (0.0625 + 0.5625) = 1 - 0.625 = 0.375$$

$$p_{\text{Yes}} = 0.5$$

$$p_{\text{No}} = 0.5$$

$$\text{Gini}_{\text{Weak}} = 1 - (0.5^2 + 0.5^2) = 0.5$$

$$\text{Gini}_{\text{Wind}} = \left(\frac{6}{10} \times 0.5 \right) + \left(\frac{4}{10} \times 0.375 \right) = 0.3 + 0.15 = 0.45$$

Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes

Ordenamos por temperatura

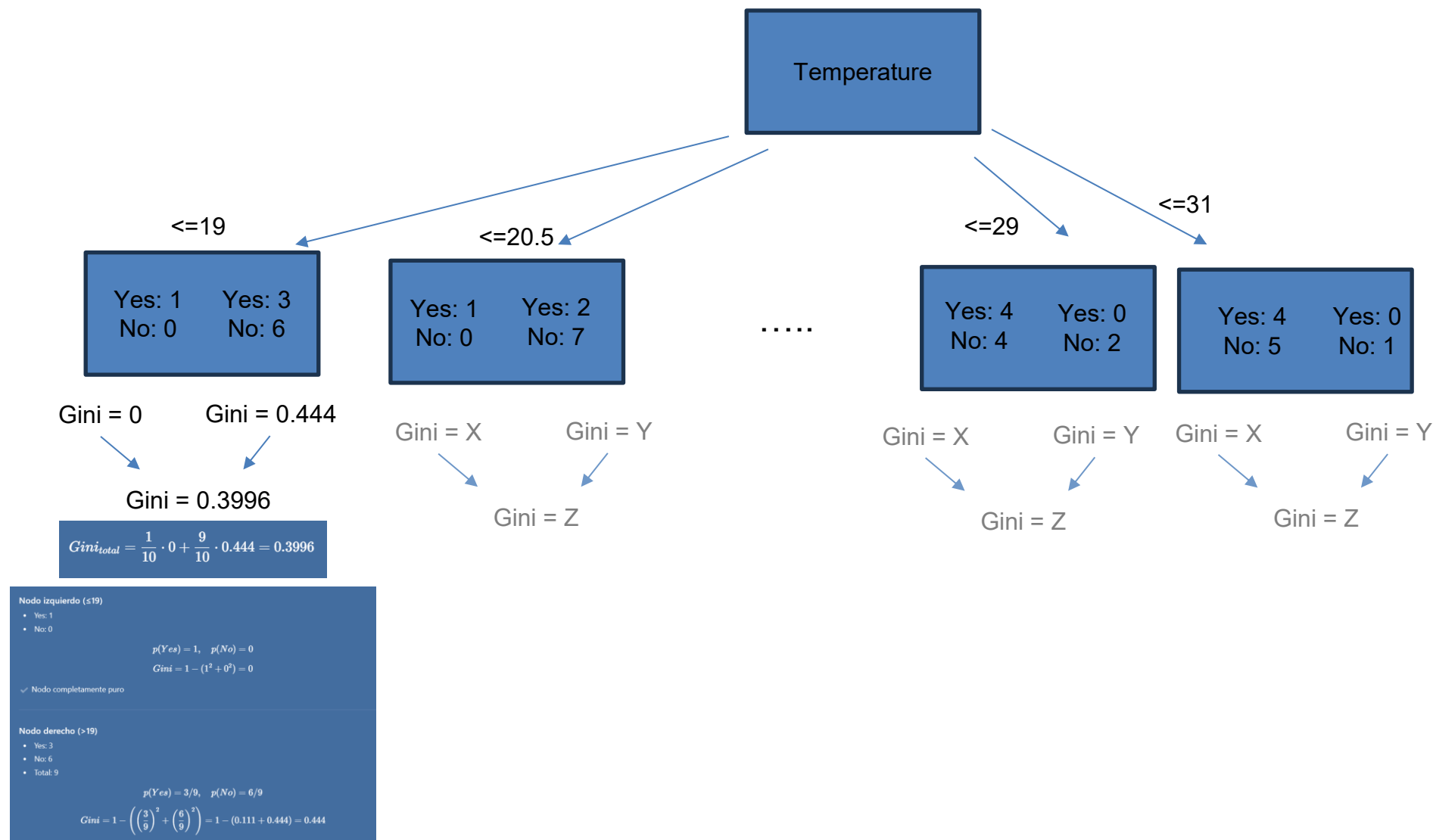
Temperature	Play Tennis
18	Yes
20	Yes
21	No
22	No
24	Yes
25	Yes
25	No
28	No
30	No
32	No

Calculamos los medios de cada tramo

Temperature	Play Tennis
18	Yes
19	
20	Yes
20.5	
21	No
21.5	
22	No
23	
24	Yes
24.5	
25	Yes
25	No
26.5	
28	No
29	
30	No
31	
32	No

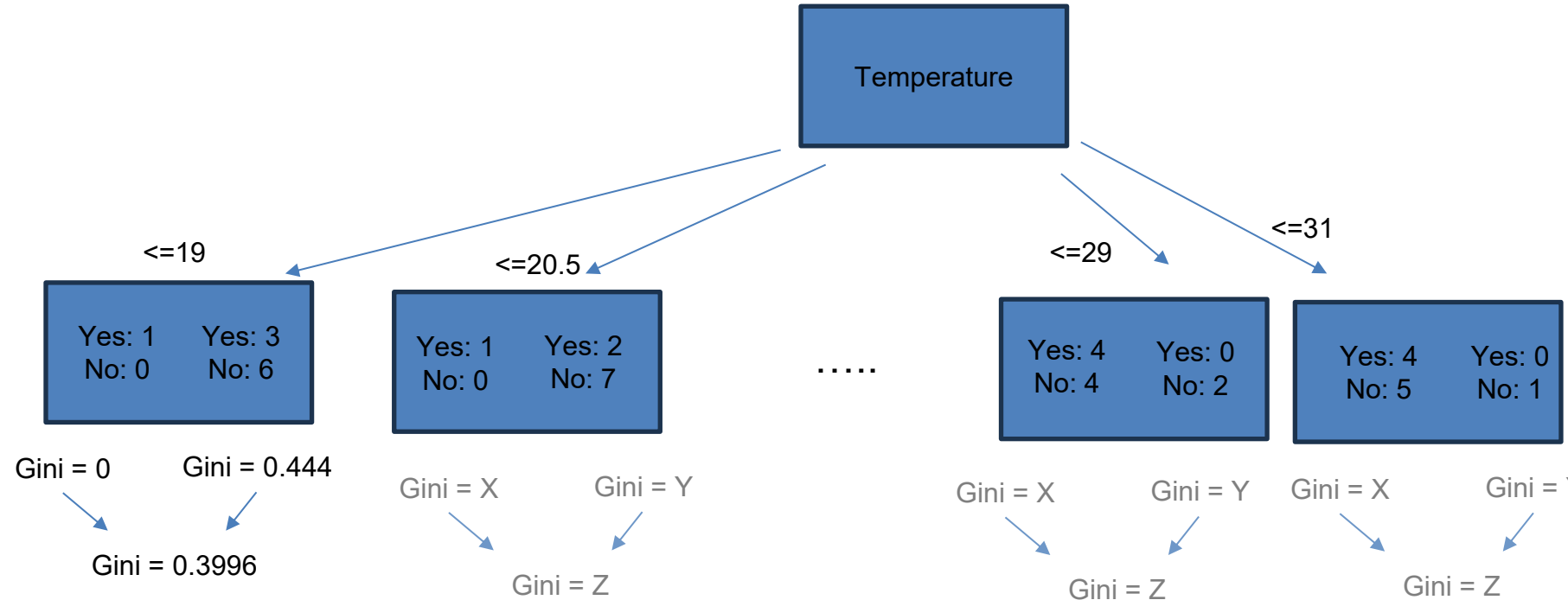
Clasificación multiclase: Árboles de clasificación

Temperature		Play Tennis
18	19	Yes
20	20.5	Yes
21	21.5	No
22	23	No
24	24.5	Yes
25	26.5	Yes
25	29	No
28	31	No
30		No
32		No



Clasificación multiclase: Árboles de clasificación

Temperature		Play Tennis
18	19	Yes
20	20.5	Yes
21	21.5	No
22	23	No
24	24.5	Yes
25		Yes
25	26.5	No
28	29	No
30	31	No
32		No

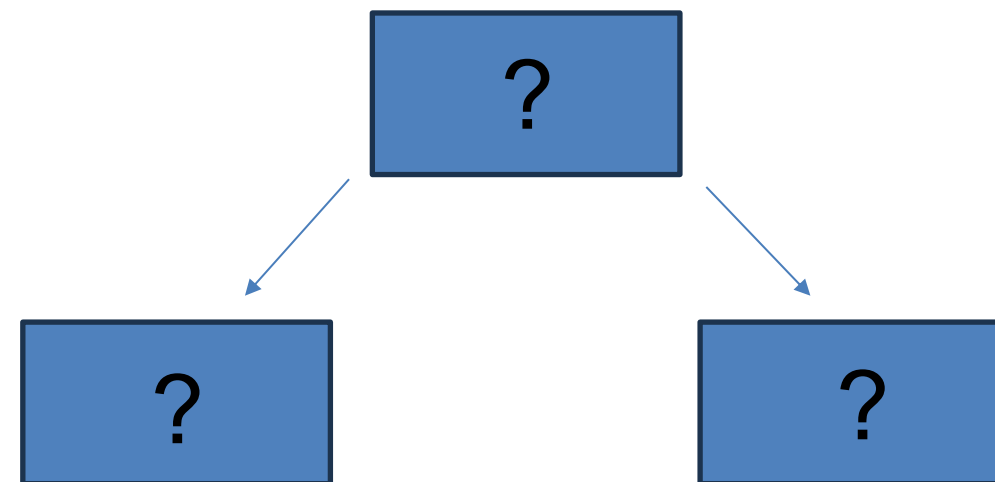


Suposición: El mínimo Gini impurity es 0.23

Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes

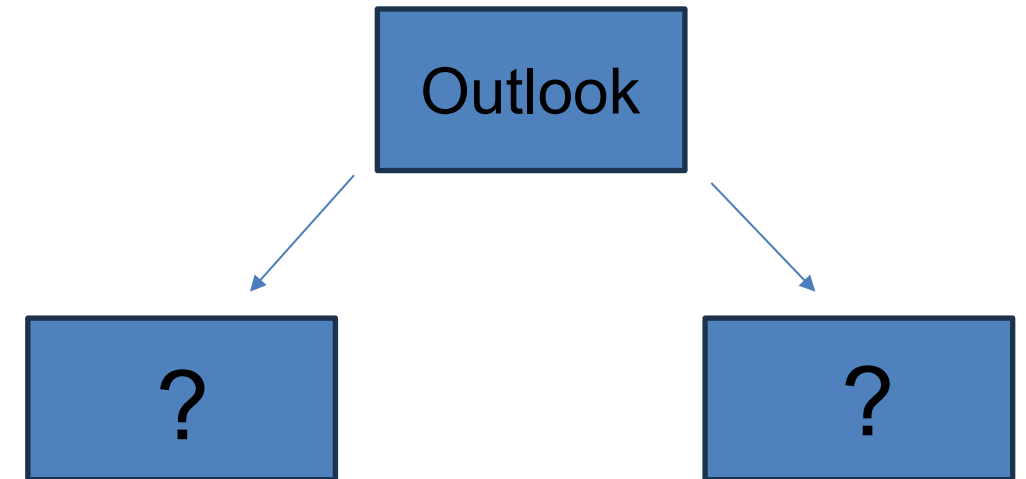
Outlook: 0.2
Temperature: 0.23
Wind: 0.45



Clasificación multiclase: Árboles de clasificación

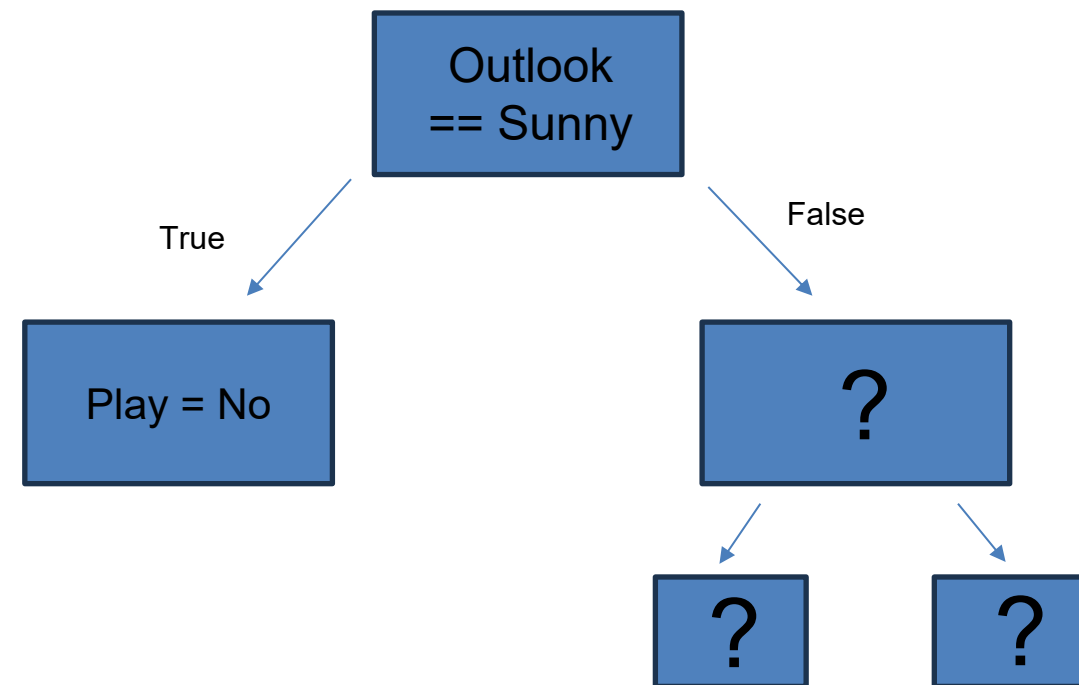
Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes

Sunny: 0
Overcast: 0
Rain: 0.5



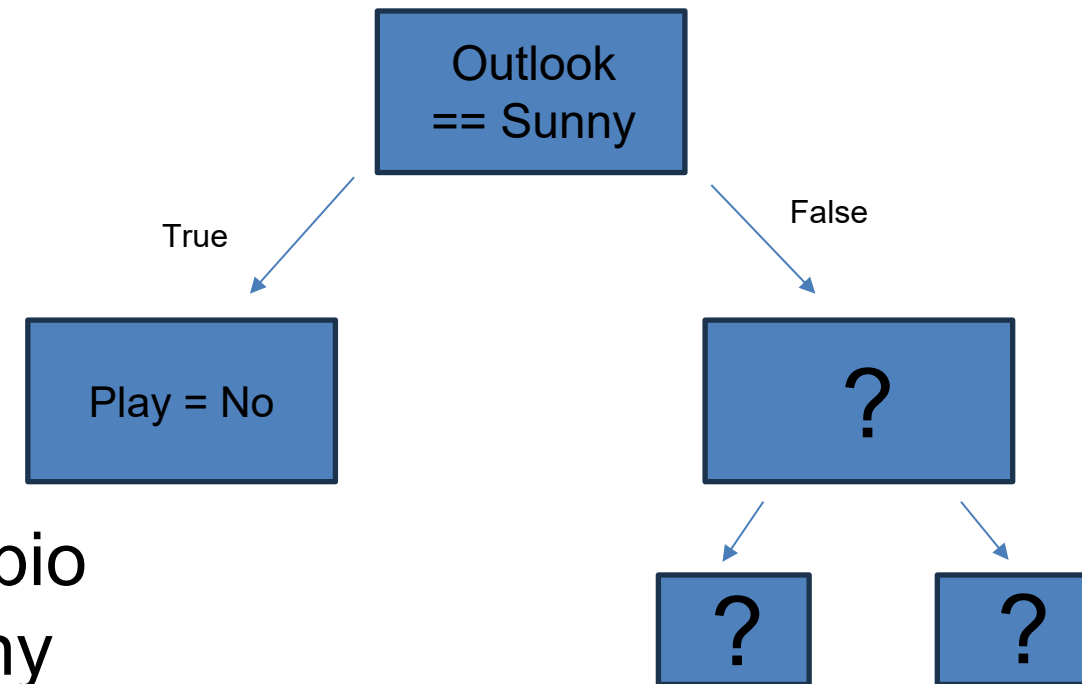
Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Sunny	30	Weak	No
Sunny	25	Weak	No
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Sunny	28	Strong	No
Sunny	32	Weak	No
Rain	18	Weak	Yes



Clasificación multiclase: Árboles de clasificación

Outlook	Temperature	Wind	Play Tennis
Overcast	25	Weak	Yes
Rain	20	Weak	Yes
Rain	22	Strong	No
Rain	21	Strong	No
Overcast	24	Strong	Yes
Rain	18	Weak	Yes



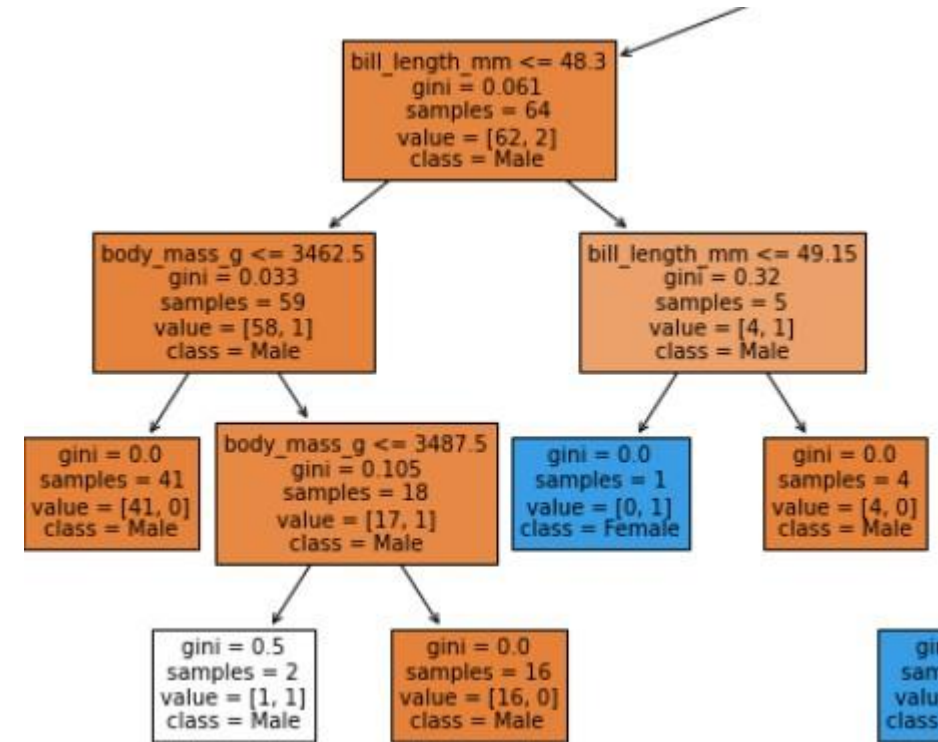
Volvemos a hacer todo desde el principio sin los elementos que Outlook = Sunny

Clasificación multiclase: Árboles de clasificación

Si dejamos que el árbol crezca de forma infinita tendremos **sobre entrenamiento (overfitting)** y no generalizaremos el conocimiento para nuevas muestras.

Podemos controlar todos estos parámetros para mejorar el modelo base pero hay que hacerlo con cuidado.

Al llegar a los nodos hoja (finales) clasificaremos las muestras en aquel que hayamos 'caído'.



Árboles de clasificación: Cuándo y por qué

La primera circunstancia para utilizar los árboles de decisión es cuando necesitamos clasificar para más de 2 clases (que no es excluyente, podemos utilizarlos para clasificación binaria) y como todos los algoritmos, tienen sus puntos fuertes y débiles:

- **Algoritmo base** para la clasificación multiclase, no tenemos muchas más opciones si queremos tener una **alta explicabilidad** (keep it simple).
- **Entrenamiento rápido y converge con facilidad.**
- Podemos utilizarlo para **selección de variables**. Al utilizar métricas de entropía y poder extraer el conocimiento adquirido de forma sencilla, se puede utilizar como una verificación de la selección de variables (junto con el análisis, correlaciones y PCA)
- Tiende mucho al **sobre entrenamiento** y es necesario ajustar los parámetros.
- Como todos los algoritmos básicos, tiene un techo relativamente fácil de alcanzar y por ello se utilizan algoritmos más complejos.



iAl código!

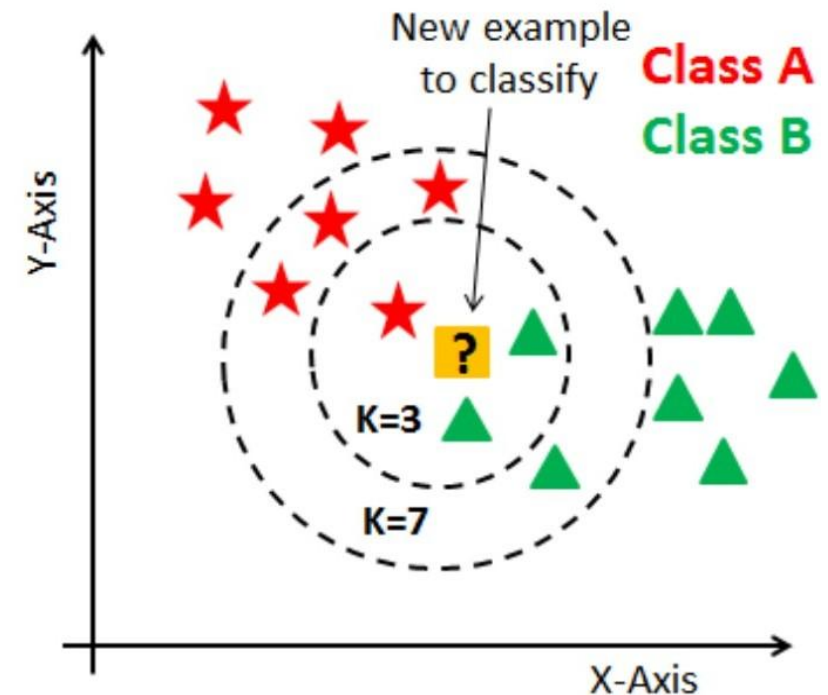
DecisionTreeCredit

Clasificación multiclase: K-NN

Tiene en cuenta la **distancia** (como tantos otros algoritmos vistos) y asigna las nuevas muestras a la clase más repetida al su alrededor. El algoritmo es muy sencillo y necesita muy pocos pasos:

- Calcular la distancia de la nueva muestra con todas las disponibles del entrenamiento.
- Las ordena de forma ascendente y se escogen los K vecinos.
- Se asigna la clase mayoritaria de los vecinos cercanos.

¿Qué **distancias** utilizamos?



K-NN: Distancias

Euclidiana

- Mide la distancia “recta” entre dos puntos en un espacio n-dimensional.
- Se calcula con la raíz cuadrada de la suma de las diferencias al cuadrado.
- Sensible a la escala de las variables, útil para datos continuos.

City Block (Manhattan)

- Suma las diferencias absolutas entre las coordenadas de los puntos.
- Representa movimiento en “rejilla” como calles de una ciudad.
- Menos sensible a valores extremos que la euclidiana.

Correlación

- Evalúa qué tan similares son los patrones de dos vectores.
- Ignora la magnitud, se centra en la relación lineal entre variables.
- Útil cuando importa la forma de los datos, no su escala.

Coseno

- Mide el ángulo entre dos vectores, no su longitud.
- Valor cercano a 1 indica vectores muy alineados (similares).
- Muy usado en texto o recomendación, donde importa dirección más que tamaño.

Euclidean
$$D = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$

City block
$$D = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$

Cosine
$$D = \frac{\sum_i x_i y_i}{\sqrt{\sum_i x_i^2} \sqrt{\sum_i y_i^2}}$$

Correlation
$$D = \frac{N \sum xy - (\sum x)(\sum y)}{\sqrt{[N \sum x^2 - (\sum x)^2]} \sqrt{[N \sum y^2 - (\sum y)^2]}}$$

K-NN: Cuándo y porqué

De forma completamente intuitiva, tiene sentido asumir que cualquier muestra en un espacio de N dimensiones será muy parecida a las que tenga cerca y por lo tanto, pertenecen al mismo grupo y etiqueta (recordemos clustering). Aún con ello, el algoritmo tiene más problemas que ventajas:

- **Recursos computacionales altos** al guardar todas las muestras para realizar las medidas.
- No reacciona bien al **ruido** ni a las variables que no aportan información (no pondera si no lo hacemos a mano)
- Utilizar funciones de distancia afecta (y mucho) al funcionamiento.
- **Poca explicabilidad** de la estructura así como un uso nulo del conocimiento que puede generar (a diferencia de los árboles de decisión)



iAl Código! KNN



Bibliografía

Bibliografía

- Apuntes regresión logística: <https://cienciadedatos.net/documentos/py17-regresion-logistica-python>
- Apuntes árboles de decisión: https://cienciadedatos.net/documentos/py07_arboles_decision_python
- Apuntes de máquinas de soporte: https://cienciadedatos.net/documentos/34_maquinas_de_vector_soporte_support_vector_machines



Carnation